

ניתוח דרישות ובדיקות סטטיות

מושגים בפרק A – סטטי מול דינמי

• **Static Testing | בדיקה סטטית** – בדיקה שמתבצעת ללא הרצת התוכנה. בכלי או ידנית בוחנים תוצרים (מפרטים, קוד, דיאגרמות) כדי לשפר איכות ולהבטיח עקביות ותקינות (verification/validation) [L199+3].
[L207]. בשונה מבדיקה דינמית, אין צורך בביצוע קוד או תרחיש. קופירייטינג K2-level: K. מבחן: זהו סוגי תוצרים (e.g). מפרטי דרישות, ארכיטקטורה, קוד, מסמכי תכנון [L233-L240+3] (שניתנים לסקירה סטטית וכיצד סטטי תומך בזיהוי תקלות כמו קוד לא נגיש, אי-עמידה בעיצוב, טעויות לוגיות במפרט [L252-L260+3] [L290+13].
[L296]. טעות נפוצה: התמקדות רק בשגיאות כתיב ותחביר במסמך במקום בתוכן הלוגי שלו.

• **Dynamic Testing | בדיקה דינמית** – בדיקה בהפעלת התוכנה בפועל (לדוגמה באמצעות הרצת test cases) כדי לוודא שהמוצר מתנהג כנדרש. מאפשרת לבדוק תקלות תפעוליות (למשל בעיות בביצועים, זיכרון, אינטגרציה) שרק מופיעות בריצה אמיתית. רמת K2. מבחן: הבדלים בין בדיקה סטטית ודינמית – למשל, סטטי ימצא קוד בלתי נגיש וטעויות במפרט שאינן מתקבלות תוך הרצה, ואילו דינמי יאתר באגים כמו נזילות זיכרון או זמן תגובה איטי שמתגלים רק בעבודה ממשית [L290-L296+13]. טעות נפוצה: לחשוב שהבדיקה הסטטית "מספיקה" לבדיקת הלוגיקה והקוד מבלי לבצע בדיקות דינמיות ממשיות, או להפך להקל ראש בבדיקות סטטיות כחסרות ערך.

• **עלות תיקון פגם לפי גילוי** – ככל שפגם מתגלה מאוחר יותר (בייצור או בסביבת לקוח), עלות תיקונו עולה באופן מעריכי. טעות דרישות שתתוקן ברוורס של מערכת או בהתקנה תנגיש עלויות גבוהות מאוד לעומת תיקונה בשלב הדרישות או התכנון [L817-L825+22]. הקשר: לכן בדיקות סטטיות של דרישות או עיצוב מוקדמות מוזילות את עלות הפרויקט [L817-L825+22]. רמת K2. מבחן: להבין שבדיקה מוקדמת (דרישות/עיצוב) עולה פחות ועלויה למנוע rework מאוחר [L817-L825+22]. טעות נפוצה: לדחות ביקורת על מפרטים עד שלאחר בניית הקוד, מתוך מחשבה ש"עוד יבוא זמן לתקן".

• **תוצרי פיתוח לסקירה סטטית** – Almost כל תוצר שניתן לקריאה והבנה ניתן להיבדק סטטית [L233+3].
[L240]. דוגמאות: מסמכי דרישות, תיקים (use cases), user stories, דיאגרמות אדריכלות, קוד מקור, מסמכי תכנון, מקרי מבחן, חוזים ועוד [L233-L240+3]. הרעיון: כל תוצר "מוחשי" שאפשר לעבור עליו ב"קריאה ובדיקה" עשוי להכיל תקלות שנתפסות בשלב סטטי (למשל חוסר עקביות בין דרישות). רמת K1. מבחן: לדעת לציין סוגי מסמכים שנבדקים (למשל מפרט דרישות, תוכנית בדיקות, מודלים) [L233-L240+3]. טעות נפוצה: להתמקד רק בקוד (Code Review) ולדלג על סקירות תכנים כגון מפרטים או User Stories.

מושגים בפרק B – פגמי דרישות

• **Ambiguity | דו-משמעות** – דרישה שאינה חד-משמעית: ניתן לפרש אותה בדרכים שונות. למשל "המערכת תהיה ידידותית למשתמש" או "מהיר" – לא מוגדרים במדויק [L817-L825+22]. הגדרה: המשמעות אינה ברורה לכל הצדדים. רמת K2. מבחן: זיהוי ניסוחים מופשטים ("מהיר", "קל לשימוש", "ידידותי") ואמביוסטים, הסבר מדוע הבעיה (כל משתמש יכול לתפוס זאת אחרת), והצעת ניסוח הבהרה מדיד (למשל: "תגובה תוך פחות משתי שניות"). טעות נפוצה: מתעלמים ממשפטים כאלה או מנסים "לתקן בראש", במקום לשאול שאלת הבהרה ולקבל דרישה ברורה.

• **Omission | חסר** – דרישה לא שלמה או חסרה פרט הכרחי. למשל לא צוינו תנאי קצה, שדות חובה, תנאים עסקיים חשובים. הגדרה: דרישה לא כוללת את כל המידע להגשמתה או לבדיקה שלה. רמת K2. מבחן: זיהוי חוסרים (למשל, דרישה להגבלת שדה ללא ציון האם התגובה היא שגיאה או חסר ברירת מחדל) והשלמתם בבקשת הבהרה. טעות נפוצה: להתעלם מפרטים חסרים במפרט (למשל משאירים out-of-scope שאלות במקום).

• **Contradiction | סתירה** – דרישה שסותרת דרישה אחרת. למשל, דרישה אחת שדורשת שמשקל ההזמנה יהיה מעל 0, ודרישה אחרת האומרת שהוא יכול להיות 0. הגדרה: קיים מצב בו הדרישות אינן עולות בקנה אחד זו עם זו. רמת K2. מבחן: מציאת זוג דרישות סותרות (למשל, שונות על ערכי מותר/אסור) והבהרת הסטנדרט הרצוי. טעות נפוצה: להתייחס לכל דרישה בנפרד, בלי לבדוק אותה מול כל שאר הדרישות.

• **Unmeasurable | בלתי-מדיד (אי-ניתן-אמינות)** – דרישה ללא קריטריון ברור לבדיקתה. למשל "המערכת תענה במהירות" או "תהיה ידידותית". הגדרה: קשה או בלתי אפשרי לאפיין אבחנה של מימוש. רמת K2. מבחן: זיהוי ניסוחים סובייקטיביים שאינם מוגדרים (למשל 'זורם', 'מצוין') והצעת מדדים או פרמטרים ברורים במקום (למשל יעד זמן תגובה מדויק, אחוזי שימושיות). טעות נפוצה: להתייחס לדרישה כתקנה מבלי לשים לב שאין דרך אובייקטיבית לבדוק אותה.

• **Solution in Requirement | ערבוב פתרון בדרישה** – דרישה שכוללת בתוכה אלמנט של פתרון טכנולוגי במקום לצייין את הצורך. למשל "המערכת תציג דו-חום באמצעות גרף עוגה" (הנחיית יישום במקום צורך עסקי). הגדרה: דרישה אינה מתמקדת במה אלא מנסחת איך. רמת K2. מבחן: זיהוי דרישות שמביאות רעיון שיטה (לדוגמה שימוש בטכנולוגיה X או כלי Y) והגדרת הדרישה מחדש לפי התוצאה הרצויה (למשל "להציג התפלגות של קטגוריות המכירות"). טעות נפוצה: אינטגרציה של הנחיות פיתוח בבקשת המערכת, שגורמת לעתים לחוסר גמישות ושינוי קשה אם יבוא פתרון אחר.

מושגים בפרק C – בדיקתיות (Testability)

• **Testability | בדיקתיות** – מדד היכולת של דרישה או מערכת להיבדק על ידי מבחנים. דרישה תקינה תהיה ברורה, מדידה וניתנת לאימות. רמת K2. מבחן: דרישה תהיה ניתנת לבדיקה אם היא חד-משמעית, מגדירה תנאים קלט ופלט ברורים וכוללת גבולות וקריטריונים. לדוגמה, במקום "המערכת תיתן תגובה מהירה" – נגדיר "זמן תגובה של עד 2 שניות" (מאפיין מדיד). טעות נפוצה: להניח שדרישה כלשהי ברורה כשאין לה קריטריונים מדידים (למשל "קל למשתמש").

• **Clarifying Question | שאלת הבהרה נכונה** – שאלה שמחדדת את הדרישה בבדקנות בונה. הגדרה: שאלה מנומסת ובהירה שמבקשת להוסיף פרטים או להסיר עמימות מהדרישה, במקום להאשים או למתוח ביקורת. רמת K3. מבחן: ניסוח שאלות כמו "כיצד המערכת תתמודד במצב שבו X?" או "מה הכוונה ב-'ידידותי' למשתמש" בהקשר הזה?" שיכולות לסייע למפרט להבהיר פרטים קריטיים. טעות נפוצה: לנסח שאלות כמשפטי הטחה ("זו דרישה אי אפשרית!") במקום לחקור את הרקע והכוונה.

• **SMART/Testable Criterion | סמרט / קריטריון בדיקה** – שימוש בעקרון SMART לדרישות: דרישות צריכות להיות *Specific, Measurable, Achievable, Relevant, Time-bound*. הגדרה: בדיקה בעזרת שאלת SMART: האם הדרישה ברורה ומדידה? רמת K2. מבחן: לוודא שכל דרישה מציגת לסמארט: ספציפית (מגדירה בדיוק מה נבדק), מדידה (ניתנת לבדיקה בתנאים מוגדרים), וכדומה. לדוגמה, נניח דרישה כפי שהיא לעומת שאלות SMART לגביה (Are all terms quantified? Is acceptance defined?). טעות נפוצה: להתרכז רק בחלק מאפייני SMART (לדוגמה ספציפיות) ולדלג על מדידות ואיפיון תוצאה, מה שהותיר דרישות לא מספקות לבדיקה.

מושגים בפרק D – סקירות ובדיקות סטטיות

• **Informal Review | בדיקה בלתי פורמלית** – סקירה מהירה ללא תהליך מוגדר: קולגה מסתכל על המוצר בקצרה (למשל בדיקה בעין של מסמך, "pull-request"). רמת K2. מבחן: ההבדל בין informal לשאר (אין הגדרות, אסטרטגיה רשמית או רישום). טעות נפוצה: להתייחס לבדיקת קוד תוך כדי עבודה יומיומית כסקירה מספקת, מבלי לבצע סקירה שיטתית של המסמך כולו.

• **Walkthrough | הליכה דרוכה** – סקירה בה היוצר של המסמך מציג אותו לשאר המשתתפים שלב-לא-שלב, מסביר את התוכן, והצוות מעלה שאלות ובעיות. רמת K2. מבחן: בהבנת תפקידו של המחבר כ"מרצה" (מוביל את הפגישה) והמשתתפים כמבקרים בעין. טעות נפוצה: לערוך walkthrough בלי נציג הדרישות או המחבר, מה שגורם לחסר הבנה ולזליגת מידע.

• **Technical Review | סקירה טכנית** – סקירה מובנית שמהווה בדיקת הנכונות הטכנית של המוצר. מבוצעת על-ידי מומחים (לעיתים ללא נוכחות מחבר המסמך), כולל הכנה אישית, רשימת בדיקות ספציפיות, ותיעוד תוצאות. רמת K2. מבחן: ידע שהתהליך רשמי יותר מ-walkthrough ופחות פורמלי מ-inspection, ומתמקד בתכנים המקצועיים (למשל עמידה בסטנדרטים טכניים) בלי נחיצות בנוכחות המחבר. טעות נפוצה: לחשוב ש-inspection זהה ל-technical review; בפועל, inspection קפדנית יותר (עם דיגל כניסה/יציאה ומדדים).

• **Inspection | סקירה פורמלית (Inspection)** – הסוג המובנה והקפדני ביותר של סקירה. כולל תפקידים מוגדרים (moderator, author, reviewer, scribe), קריטריוני כניסה/יציאה, מדדים על כמות התקלות, וחייב היערכות מקדמית. רמת K2. מבחן: להכיר שמודולטור מנהל את התהליך, הסוקר (author) מעלה את המסמך, וכל בעיה מתועדת על-ידי המזכיר; התהליך בנוי ממספר שלבים (תכנון, חלוקת חומר, סקירה אישית, פגישת סקירה, שיפור ותיקון, בדיקת [22†L874-L883] [22†L861-L870] follow-up). טעות נפוצה: לא להקפיד על הכנה אישית לקראת inspection – לפעמים בודקים נכנסים לפגישה מופתעים ולא מוכנים, מה שמקטין יעילות התהליך.

• **תפקידי סקירה** – רשימת תפקידים בסקירה פורמלית: **Author** (מחבר/יוצר המסמך), **Moderator** (מנחה/מתווך), **Reviewers** (בוחנים), **Recorder/Scribe** (מקליד/מתעד), **Manager** (מחליט על קיום הסקירה) [L849-L858†22]. רמת K1. מבחן: לדעת מה תפקידו של כל אחד, למשל: Moderator מארגן ובוחן האם הקריטריונים מתקיימים; Author משיב על שאלות ומתקן; Scribe מתעד. טעות נפוצה: לחשוב שמנהל הפרויקט משתתף בפגישה – בדרך כלל הוא רק מאשר ומשחרר זמן, ולא בהכרח נוכח בפועל [L849-L858†22].

• **תהליך סקירה פורמלי (חמשת השלבים)** – התהליך כולל (1) **תכנון** – Moderator קובע מטרות, בוחר מבקרים וקובע קריטריוני כניסה/יציאה; (2) **קיק-אוף** – פגישה קצרה לחלוקת מסמכים והסבר מטרות; (3) **סקירה עצמאית (Individual Review)** – כל אחד בוחר ועובר על המסמך בעצמו [L870-L878†22]; (4) **פגישת סקירה** – דיון ודווח על ממצאים, הסבר על כל פגם, וכל בעיה מתועדת על ידי (5) [22†L874-L883] Scribe **שיפור (Rework)** ובדיקה חוזרת (**Follow-up**) – Author מתקן את התקלות, Moderator בודק שהכל הושלם וקריטריוני היציאה (exit) הושגו [L878-L883†22]. רמת K2. טעות נפוצה: ניסיונות לפתור בעיות בזמן הפגישה במקום לתעדן; ההמלצה היא שהדיון לפתרון יהיה לאחר הבדיקה המקדימה [L884-L892†22].

• **גורמי הצלחה בסקירה** – התנאים להפוך סקירה יעילה: הגדרת מטרות ברורות ובקרה על השגתן; בחירת מבקרים מתאימים (למשל מומחי תוכן/אבטחה לפי נושא) [L893-L900†22]; שמירה על ביקורת עניינית (לא אישית) ושיפור אווירה (מניעת התנגדות מוגזמת) [L899-L907†22]; תמיכה ניהולית של הפרויקט (הקצאת משאבים וזמן ייעודי); הכשרה בטכניקות סקירה לכל המעורבים [L904-L909†22]. רמת K3. מבחן: ניתוח תרחישים – למשל, אם מבקרים לא בקיאים בנושא, הסקירה עלולה לחמוק מתקלות; או אם משאבים קצרים, הסקירה נעשית חפוזה וחסרה עומק. טעות נפוצה: למקד את הביקורת בסגנון או בפן טכני בלבד, במקום לוודא שהבאגים הלוגיים אכן מתועדים ומתוקנים [L893-L902†22].

מושגים בפרק E – קריטריוני קבלה

• **Acceptance Criteria | קריטריוני קבלה** – הגדרה: תנאים או דרישות מיוחדים ש-Feature או User Story חייב לעמוד בהם על מנת שהדרישה תיחשב "הושלמה". מבנה נכון: בדרך כלל רשימת תנאים ברורים, אפשר לרשום כבדיקות (Given-When-Then וכדומה). רמת K2. מבחן: דרישות הסף הן קריטריונים ברורים ומדידים, המשמשים כ"לוח זמנים" להחלטת קבלה של דרישה. לדוגמה, במקום "מערכת תהיה מהירה", ננסח "בעת חיפוש מוצר, רשימת התוצאות תוצג תוך ≥ 2 שניות". טעות נפוצה: השארת מילים כלליות ("מהיר", "ידידותי", "מתאים לכל המסכים") שאינן ניתנות לאימות [L6-L10†43].

• **Given-When-Then (GWT)** – תבנית נפוצה לכתיבת acceptance criteria:

- **Given** (בהינתן) – תנאי פתיחה או מצב התחלתי,
- **When** (כאשר) – הפעולה שהמשתמש עושה,

• **Then (זא) – התוצאה הצפויה (התנהגות המערכת).**
 פורמט זה ממקד את הדרישות בתסריטים ברורים. רמת K2. מבחן: זיהוי רכיבי ה-GWT בכל קריטריון. למשל:
 "בהינתן שמשמש מחובר, כאשר ילחץ שמירה, אז יופיע מסך אישור." טעות נפוצה: כתיבה של קריטריון בתצורה חופשית (בדומה למשפט רגיל) בלי הפרדת תנאים, מה שמקשה על הפיכת הקריטריון לאבני דרך למדידה והאוטומציה.

• **קריטריון בלתי ניתן לאימות (לא-מדיד) –** קריטריון קבלה מנוסח באופן שאינו מאפשר בדיקה אובייקטיבית. למשל: "המנהל יהיה מרוצה ממידע הדוחות". זיהוי: חיפוש מילים סובייקטיביות (כמו "מרוצה", "מהיר", "קלה"). רמת K2. מבחן: הצגת משפט כזה ובקשה לפרט מדד או להסיר את הבלתי מדויק. דוגמא: אם כתוב "המערכת תהיה מהירה", אפשר לשאול – "מהו זמן תגובה מקובל?" [L6-L10†43]. טעות נפוצה: לפרש את הקריטריון לפי השיפוט האישי של המבקר במקום לדרוש הבהרה (למשל לקבל הנחיה חדשה: "Response time ≤ 2s").

טעויות נפוצות בבדיקת דרישות ובדיקות סטטיות

- **מתקנים בראש ולא שואלים –** מבצע הסקירה מנחש את התשובות בעצמו במקום לפנות למבצע הדרישות לבירור (זוהי טעות, לפעמים גוררת תיקון שגוי או לא-ממוסמך).
- **התמקדות בסגנון ולא בתוכן –** סוקרים מטים לבקר שגיאות ניסוח, התכתבויות אנגלית וכו' במקום להתרכז בפגמים לוגיים ומבניים בדרישה.
- **אין הכנה לקראת סקירת Inspection –** העובדה שלא קראנו באופן אישי את המסמכים לפני הפגישה מובילה לבזבוז זמן ודיון לא יעיל. בסקירה פורמלית מצופה שכל מבקר יכין את עצמו מראש.
- **סיום מדי מוקדם של סקירה –** סוקרים עשויים להגיע לנקודת "אוש" ("כבר בדקנו הכול") ולפספס תתי-דרישות. חשוב לוודא שכל סעיף נבדק.
- **שיפוט מוקדם של דרישות –** מצבים בהם בודקים קובעים שדרישה מוגזמת או טכנית מדי, במקום לבחון את ההשלכות ולהגיש שאלות.
- **בחינה של הכללים במקום דרישות –** לעיתים בסקירות קוד (code review) מתמקדים רק בסטנדרטים (סגנון קוד) במקום לבדוק את הלוגיקה העסקית שבקוד.
- **סדר-חריג בתעודת פגמים –** להעדיף באגים לפי "מי צועק יותר" ולא לפי השפעה ממשית או חומרה אמיתית.
- **חלופות תעודת לא מנומקות –** החלטות במשיבוץ בעיות שנעשות באופן אקראי או ללא הסבר (e.g., "המפתח אמר כן").
- **פרמטרים בלתי מדידים למדידת התקדמות –** שימוש במדדים מסולפים לפיקוח (למשל מדד כיסוי לפי אורך מסמך) שלא משקפים כראוי מצב המוצר.
- **Exit criteria לא אובייקטיביים –** ניסוח קריטריון יציאה כמו "כל הבדיקות בוצעו" ללא הגדרה מה נבדק ומתי זה "מוכן".
- **אינטגרציה של אוטומציה בלי בדיקת ידני –** לעיתים מפתחים מטמיעים בדיקות סטטיות (לינטר וכו') ומניחים שכל השגיאות ייתפסו, בעוד שבדיקות ידניות יצריכו בדיקה מעמיקה.

מבני תרגיל לדוגמה

• **תרגיל A: מציאת פגמים בדרישת מערכת.** ינתן קטע דרישה קצר בעברית הכולל מספר שגיאות מוסוות (במקום דרישות תקניות). הנבדק יתבקש לזהות כל פגם, לסווגו (לדוגמה: דו-משמעות, אי-מדידות, סתירה, חסר, פתרון), ולנסח מחדש כל סעיף כך שיהיה ברור ובדוק. למשל:

קטע לדוגמה: "בעת לחיצה על כפתור 'שלח' הדואר האלקטרוני יישלח מיד לכל הנמענים מבלי שתתרחש שגיאה."

פגמים: "מיד" (לא מדיד), "ללא שגיאה" (בלתי ניתן לאימות), "לכל הנמענים" (חסר פירוט – מה קורה אם אין אינטרנט?) וכו'.

• **תרגיל B: שכתוב קריטריון קבלה באופן מסודר.** ינתנו מספר תרחישי User Story עם קריטריוני קבלה מנוסחים חופשי. המטלה: להעביר כל קריטריון לפורמט Given-When-Then ברור, ולבדוק אם הוא ניתן לבדיקה.

- למשל: מ-“המערכת תסנן את התוצאות באופן חכם” → ל-“Given רשימת מוצרים, When המשתמש יקליד מילת חיפוש, Then מוצגים אך ורק מוצרים התואמים את המילה”.
- **תרגיל C: מיון פגמים לפי קטגוריה.** הציגו פסקת דרישה המכילה כמה משפטים פגומים; המטלה: לסמן ליד כל משפט את סוג הפגם (דו-משמעות, חסר וכו’). דוגמה: “יש להציג ללקוח מחיר בסכום שלם שקל בלבד.” (סימון: חסר – לא מצוין מה יקרה בשקלים עם אגורות; אולי אמור להיות rounding).
 - **תרגיל D: עיצוב שאלות הבהרה.** יוצג קטע דרישה שהינו לא שלם או כללי; על המועמד לנסח 3-4 שאלות ממוקדות שימקדו את דרישות הלקוח. לדוגמה, על קטע “המערכת תשלח התרעה בזמן אמת על חריגות” ניתן לשאול: “מהו טווח הזמן המוגדר כזמן אמת?” האם ההתרעה תישלח בדואר אלקטרוני/אפליקציה? מהו פירוש החריגה בפועל?”
 - **תרגיל E: זיהוי קריטריון בלתי ניתן לאימות.** ניתנת רשימת קריטריוני קבלה מנוסחים, כאשר חלקם כוללים ניסוחים לא מדידים. על המטלה לזהות את הפסוק שאינו ניתן למדידה ולהסביר למה. למשל: “המערכת תהיה קלה לתפעול” – מדובר בעמימות (לא ניתן למדוד מתי מערכת תהיה יקלה).
 - **תרגיל F: טריאז' פגם בפיצ'ר.** תרחיש שבו מתקבל באג או התרעה מערכת (למשל, CI אדום), והבודק צריך לקבל החלטות תעדוף (Severity vs. Priority) על בסיס ההקשר והנתונים הזמינים – כולל נימוק הפוך בין שתי החלטות תקפות. לדוגמה: בחירה האם לתקן קודם קריסת מערכת חמור או בעיה פחות קריטית שמשבשת פיצ'ר עיקרי, תוך נימוק (בישראל השיקול יתמקד בשימושיות ומספר המשתמשים הפגועים).

סגנון שאלות מבחן ומלכודות טיפוסיות (פרק 3)

שאלות מיצ"ב של פרק 3 נוטות לכלול הבחנה בין סטטי לדינמי, תיאור מקרים של סוגי סקירות, ותשובות מרובות מעורפלות. מלכודות אופייניות כוללות:

- **מידת מעבר בין בדיקה סטטית ודינמית** – למשל שאלה מי מוביל את הסקירה (סטטי) או באיזה שלב מתקיימת סקירת מפרט (לסלוח), בין לבין ביצוע בדיקת קוד בפועל (דינמי).
- **מפענחי הסחות דעת בניסוח התפקיד** – בשאלות תפקידים בסקירה יתנו כתובות כמו “המפתח מנהל את הסקירה” וצריך לזהות שהזים (author) הוא בדרך כלל מי שיצר את המסמך, לא חייב המפתח.
- **שאלות על reviews משולבים** – לדוגמה “מי יכול לבצע סקירת דרישות?” כשהמענה הנכון הוא שכל בעלי העניין (אנליסט, בודק, בעל המוצר) יכולים, אבל הסחת דעת: שהמבצע צריך להיות בודק רק.
- **מלכודות בתשובות GWT** – סימון Then-I Given לא במקום או הפוך; שאלות שמציגות שלושה נתונים והדורשות לזהות באיזה חלק הוא שבהינתן או בתוצאה.
- **שאלות על תהליכי בדיקה או exit criteria** – למשל הצגת exit criterion מעורפל לבחירה כלא נכון (All). e.g. “tests executed” ללא הגדרה איך זה נמדד) מול אחד ברור ומדיד.
- **ניסוח מבלבל של קריטריונים** – כמה אופציות נראות דומות (למשל “המערכת תאפשר גישה לאחר התחברות” לעומת “גישה מתבצעת רק לאחר התחברות”), והנבדק צריך לזהות איזו משקפת פגם דרישות או אי-מיצוי בדיקה סטטית.

הקשר ישראלי

בישראל, בראיונות QA מקובל לשאול שאלות scenario-based על ניתוח דרישות ובדיקות סטטיות. למשל: “תן דוגמה לדרישה דו-משמעית שאיננה פתורה, ואיך פתרת אותה?” או “תאר כיצד היית מבצע סקירת דרישות במפגש triage”. המונחים הנפוצים בבדיקות כאן הם: **בדיקה סטטית** (סקריפט או סקירה), **בדיקה דינמית**, **סקר קוד (Code Review)** כתחליף ל-“Review” סטטי קוד, ו**קריטריוני קבלה (Acceptance Criteria)**. רבים משתמשים במונחי BDD כמו *Given/When/Then* (בתרגום: בהינתן/כאשר/אז) לרישום קריטריונים קבלה. בפורומים ישראליים (כמו קבוצות QA בגייטק וברדיט הישראלי) נדיר למצוא פירוט בתכנים, אבל testers מנוסים מבליטים בתשובותיהם את החשיבות בשאלות ברורה ובהנמקה מקצועית בעת קביעת חומרת ותעדוף בעיות (לדוגמה בפגישות triage).

מקורות

- **Foundation Level Syllabus v4.0.1, Section 3: Static Testing** – ISTQB (ASTQB) (שימוש בטרמינולוגיה וסידור מושגים) [L199-L207+3] [13+L290-L296]. רישיון: *proprietary* (תקן ISTQB).

- **Course Notes Foundations:BBST** – Kaner, Fuld & Associates , פרק 3 (סקירת דרישות ובדיקות סטטיות) [L817-L825+22] [22+L831-L839] . רישיון: CC-BY (התייחסות).
 - כותבים בתחום אבטחת איכות ו-Agile – Agile Alliance Glossary (Given-When-Then) [26+L381-L389] ; Atlassian Guide on Acceptance Criteria (מילת מפתח: "verifiable") [43+L6-L10] . רישיון: תוכן ציבורי/לא ברור (יש לציין CC-BY כשיש).
 - מדריך אקדמי/מלמד – "CTFL Study Guide – Static Testing" (מקורות מצוטטים מתוך סקירה על Static vs Dynamic ולוגיקת עלות) [L817-L825+22] [22+L831-L839] . רישיון: בלתי ברור (למדפיס).
 - Community/Q&A – בלוגים ופורומים (Ministry of Testing, QA blogs) – רעיונות נפוצים על דרישות וקריטריונים (ללא ציטוט ישיר). רישיון: תוכן CC-BY או נספח (בכל מקרה נכתב במילים שלנו).
-